МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Балтийский государственный технический университет «ВОЕНМЕХ» им. Д.Ф. Устинова»
(БГТУ «ВОЕНМЕХ» им. Д.Ф. Устинова)

Кафедра Н2 Программная инженерия и интеллектуальные системы
шифр наименование кафедры, по которой выполняется работа
Дисциплина **Технологии распространения, развёртывания и сопровождения ПО**
наименование дисциплины

УЧЕБНО-ПРАКТИЧЕСКАЯ РАБОТА
№4
номер задания (при наличии)

Реализация Continuous Deployment с мониторингом и использованием Kubernetes
Вариант 17
при наличии указать тему учебно-практической работы и (или) номер варианта

ОБУЧАЮЩИЙСЯ
группы О727Б
Тарасевич В.В.
подпись фамилия и инициалы
дата сдачи

ПРОВЕРИЛ
Мажайцев Е.А.
подпись фамилия и инициалы
Оценка / балльная оценка
дата проверки

г. Санкт-Петербург
2026 г.

СОДЕРЖАНИЕ

1 Постановка задачи 3
2 Выполнение работы 4
ПРИЛОЖЕНИЕ А Листинг кода 9

1 Постановка задачи

Требуется реализовать этап Continuous Deployment созданного ПО с использованием Kubernetes и средств мониторинга. Для достижения этой цели требуется выполнить задачи, представленные ниже.

Дополнение разработанного ПО – ввиду специфики разработанного ПО (консольная утилита) требуется обеспечить его непрерывное исполнение на неопределённый промежуток времени.

Внедрение Kubernetes в проект – требуется внедрить в проект облегчённый движок Kubernetes – Minikube, создать для него манифесты Deployment и Service. Создать Helm Chart на основе созданных манифестов и развернуть приложение. В дашборде Minikube и в интерактивном режиме наблюдать успешную развёртку приложения.

Внедрение систем мониторинга – требуется развернуть в кластере Minikube систему мониторинга Prometheus, дополненную системой визуализации Grafana. Извлечь метрики из работающего приложения в обеими средствами.

2 Выполнение работы

Прежде всего, для обеспечения непрерывной работы программы в кластере minikube, был создан отдельный докер-образ, который помимо разработанного ПО включает в себя скрипт для вызова программы в бесконечном цикле с различными параметрами. Скрипт представлен на рисунке 1.

```
deralord@UbuntuDevops:~/Desktop/devops/cicd/minikube/sentrim-chart$ cat ../sentrim-service.sh
#!/bin/bash

inputList=("omg. this is my first input. that is crazy work man." "tis a senond. string." "lol third")

while true; do

        for input in "${inputList[@]}"; do
        echo "----------------------------"
        echo "input : $input"
        temp=$(sentrim "$input")
        echo "output: $temp"
        sleep 1.0

        done

done
```

Рисунок 1 – Сервисный скрипт для программы

Следуя руководству на официальном сайте, на машину был установлен движок minikube и инструмент командной строки kubectl для взаимодействия с Kubernetes-кластером. Были разработаны манифесты Service и Deployment, текст которых представлен в приложении А. Был установлен инструмент управления конфигурацией Helm и собран чарт. Вид рабочей директории представлен на рисунке 2.

```
minikube
├── Dockerfile.minikube
├── sentrim-chart
│   ├── Chart.yaml
│   ├── templates
│   │   ├── deployment.yaml
│   │   ├── _helpers.tpl
│   │   └── service.yaml
│   └── values.yaml
└── sentrim-service.sh
```

Рисунок 2 – Рабочая директория с minikube

С помощью Helm был произведён тестовый деплой приложения в кластер. Результат представлен на рисунке 3.

```
deralord@UbuntuDevops:~/Desktop/devops/cicd/minikube/sentrim-chart$ helm install sentrim .
NAME: sentrim
LAST DEPLOYED: Fri Feb 27 03:02:03 2026
NAMESPACE: default
STATUS: deployed
REVISION: 1
DESCRIPTION: Install complete
TEST SUITE: None
```

Рисунок 3 – Деплой программы через Helm

Для проверки успешности деплоя, в веб-браузере был открыт дашборд minikube, что представлено на рисунке 4.

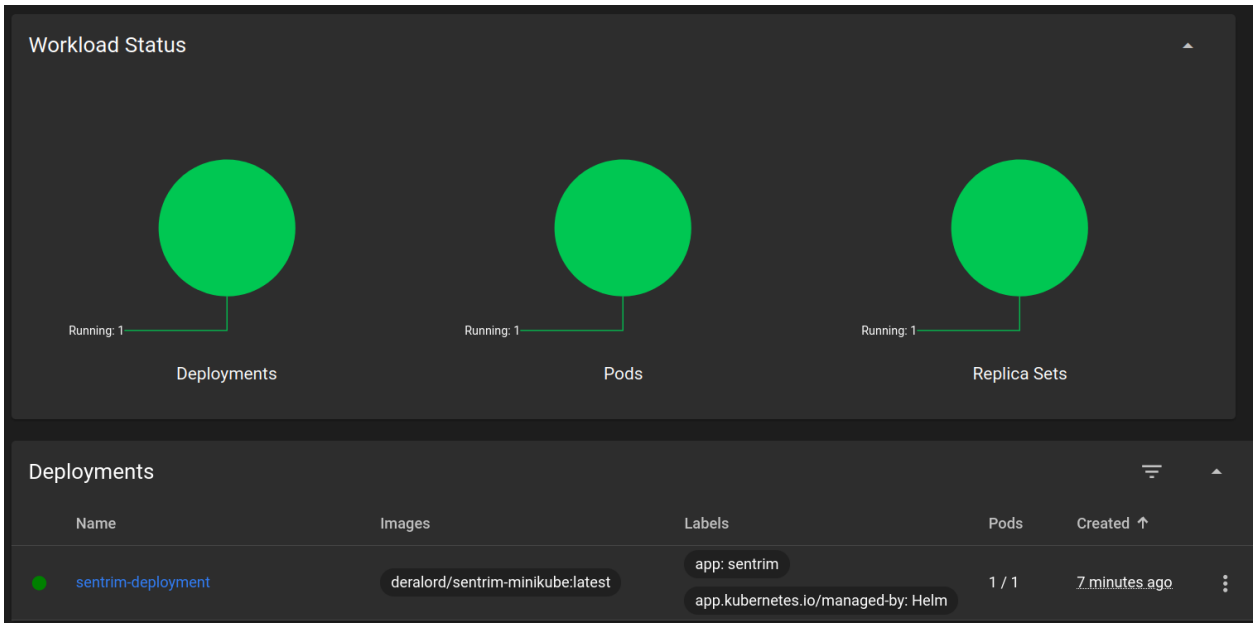

Рисунок 4 – Дашборд minikube

Из рисунка 4 видно, что в minikube развёрнут один деплой с приложением. Программа успешно выполняется в бесконечном цикле. На рисунке 5 представлен вывод в интерактивном режиме.

```
deralord@UbuntuDevops:~/Desktop/devops/cicd/minikube/sentrim
error: Unable to use a TTY - container sentrim-container did
All commands and output from this session will be recorded i
ompt.
If you don't see a command prompt, try pressing enter.
----------------------------
input : tis a senond. string.
output: senond. string.
----------------------------
input : lol third
output: third.
----------------------------
input : omg. this is my first input. that is crazy work man.
output: omg. this first input. that crazy work.
----------------------------
input : tis a senond. string.
output: senond. string.
```

Рисунок 5 – Выполнение программы в кластере

После успешного деплоя, в кластер была внедрена система мониторинга Prometheus, что представлено на рисунке 6.

```
deralord@UbuntuDevops:~/Desktop/devops/cicd/minikube/sentrim-chart$
etheus.service.type=NodePort --set grafana.service.type=NodePort --
NAME: prom
LAST DEPLOYED: Fri Feb 27 03:13:44 2026
NAMESPACE: default
STATUS: deployed
REVISION: 1
DESCRIPTION: Install complete
TEST SUITE: None
```

Рисунок 6 – Установка Prometheus

После установки Prometheus, стали доступны его веб-сервисы, что представлено на рисунке 7.

```
deralord@UbuntuDevops:~/Desktop/devops/cicd/minikube/sentrim-chart$ ku
NAME                                      TYPE        CLUSTER-IP
alertmanager-operated                     ClusterIP   None
kubernetes                                ClusterIP   10.96.0.1
prom-grafana                              NodePort    10.99.109.192
prom-kube-prometheus-stack-alertmanager   ClusterIP   10.111.23.127
prom-kube-prometheus-stack-operator       ClusterIP   10.98.18.212
prom-kube-prometheus-stack-prometheus     NodePort    10.96.15.213
prom-kube-state-metrics                   ClusterIP   10.102.234.183
prom-prometheus-node-exporter             ClusterIP   10.109.4.27
prometheus-operated                       ClusterIP   None
sentrim-service                           NodePort    10.109.185.51
```

Рисунок 7 – Сервисы Prometheus

Для просмотра и извлечения метрик, следует обратиться к prom-kube-prometheus-stack-prometheus, представляющий из себя главную панель с доступом к query-запросам, TSDB и многому другому. Так, с помощью запроса была получена информация о средней загрузке процессора за некоторый срок, что представлено на рисунке 8.

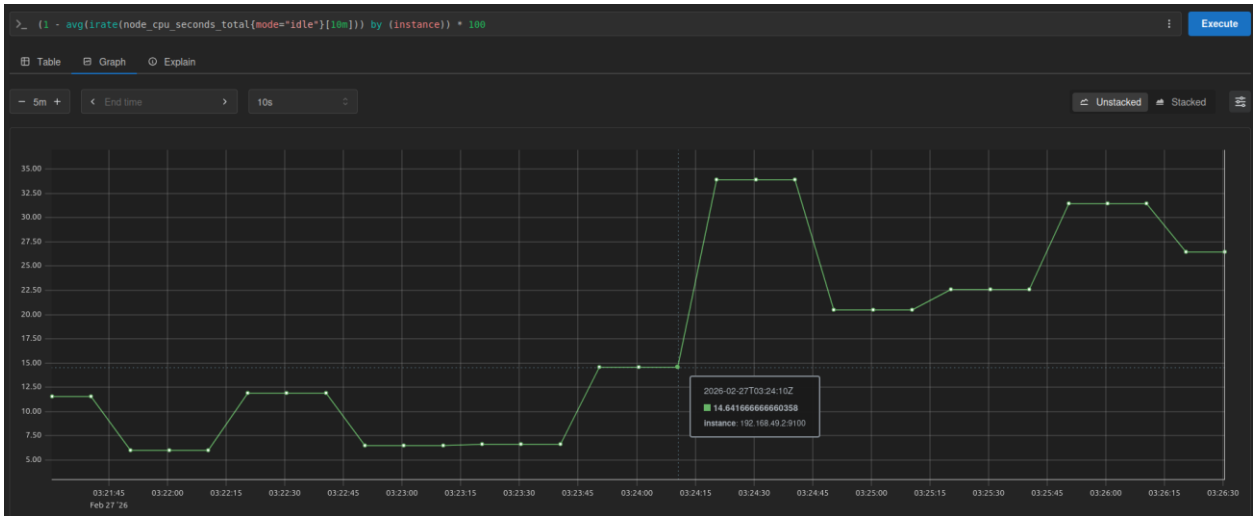

Рисунок 8 – Метрики приложения (загрузка ЦП)

Помимо встроенного в Prometheus механизма, вместе с ним поставляется Grafana, в которой можно создавать собственные дашборды или использовать предопределённые. С помощью Grafana был получен доступ к аналогичной метрике, что представлено на рисунке 9.

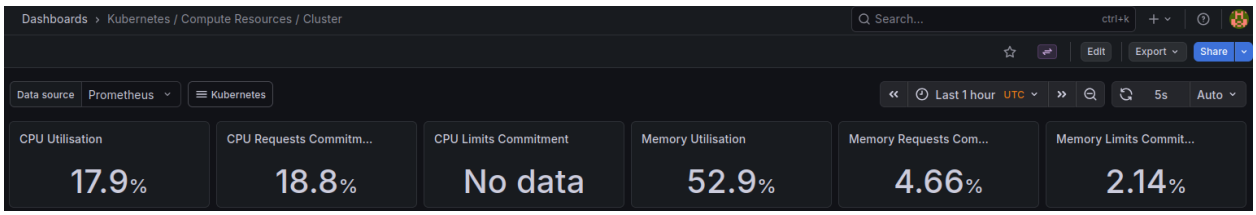

Рисунок 9 – Дашборд Grafana

ПРИЛОЖЕНИЕ А

Листинг кода

Ниже представлен листинг файла deployment.yaml.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}-deployment
  labels:
    app: {{ .Release.Name }}
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      app: {{ .Release.Name }}
  template:
    metadata:
      labels:
        app: {{ .Release.Name }}
    spec:
      containers:
      - name: {{ .Release.Name }}-container
        image: {{ .Values.image.name }}:{{ .Values.image.tag }}
        imagePullPolicy: IfNotPresent
        ports:
        - containerPort: 80
```

Ниже представлен листинг файла service.yaml.

```
apiVersion: v1
kind: Service
metadata:
  name: {{ .Release.Name }}-service
spec:
  selector:
    app: {{ .Release.Name }}
  type: NodePort
  ports:
    - port: 8080
      targetPort: 80
```